PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview**Error! Bookmark not defined.**2. Background & Business Objective 53. Product Purpose 74. System Functionalities 85. User Stories & Use Cases 156. Features Included (Scope Definition) 177. Features Not Included (Scope Control) 188. Assumptions & Constraints 199. Risks & Questions Throughout Development 21

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: RetOps

Version: 1.0

1. Project Overview

The proposed system - RetOps is a demand prediction platform that enables users to upload transactional data (mandatory) and inventory data (optional) to generate accurate demand forecasts. The goal is to help businesses optimize inventory planning, reduce stockouts, and minimize overstocking.

- **Problem Statement:** Business owners in Malaysia including sundry shops, hardware stores, restaurant owners and specialty retailers manage inventory manually through intuition and experience, with no access to affordable demand forecasting tools. They rely on manual estimation or simple heuristics. This results in recurring deadstock that locks up working capital and stockouts that directly lose sales. Despite having years of transaction records, they lack the analytical capability to extract actionable purchasing decisions from their own data.

- **Target Domain:** Retail, scaling up business owners businesses and optimizing their management and providing demand prediction. The platform specifically targets non-technical operators who hold transaction records in CSV or Excel files but currently derive no forecasting value from them.
- **Proposed Solution Summary:** RetOps enables any business owner to upload their existing sales and stock data in whatever format they have it and immediately receive a 7-day demand forecast, a concrete reorder shopping list with quantities and reasoning, and the ability to model scenarios such as upcoming public holidays or rainy weather. The GLM reasoning engine interprets the data, handles inconsistencies, and outputs its logic in

© UMHackathon 2026
Email: umhackathon@um.edu.my

plain language, transforming raw records into purchasing decisions without requiring any technical expertise from the user.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** Business owners in Malaysia tend to manage their inventory manually and often conduct through paper-based tools which is obsolete in this digital era. This can make tracking of records and demand prediction difficult due to its inconsistencies in keeping data. Additionally, many small and medium businesses lack the tools to effectively utilize their transactional data for demand forecasting, resulting in inefficient inventory management and financial loss. Seasonal and event-driven demand spikes especially during public holidays, Ramadan, school holidays, and weather shifts are predictable in hindsight but consistently catch operators off guard, resulting in either wasted perishable stock or lost revenue from empty shelves. The manual forecasting approach, based on experience and intuition, is not scalable and is prone to human error as product catalogues grow.

- **Importance of Solving This Issue:**
  - Scales inventory decision-making without requiring additional workforce
  - Removes reliance on intuition-based ordering, which is inconsistent across staff and shifts
  - Cost efficient because it eliminates the need of subscribing to third party ERP or inventory planning software
  - Reduces deadstock and working capital lockup by aligning procurement to forecasted demand

© UMHackathon 2026

- **Strategic Fit / Impact:**

  RetOps aligns directly with UMHackathon 2026's emphasis on practical GLM integration for real-world business problems. The product demonstrates three distinct AI capabilities that judges evaluate: unstructured data interpretation (handling messy CSV/Excel uploads), multi-step reasoning (forecast + recommendation pipeline), and contextual adaptation (What-If scenario planning). By targeting the retail segment, RetOps addresses a high-frequency, high-impact operational problem rather than a niche use case.

© UMHackathon 2026

3. Product Purpose

3.1. Main Goal of the System:

- To provide a digital inventory management tool for retail owners
- Generate accurate demand predictions with minimal user input - upload data, receive insights
- Reduce manual forecasting effort and reliance on intuition-based procurement decisions
- Improve inventory decision-making accuracy through data-driven, explainable AI outputs
- Enable actionable insights from existing transactional datasets without requiring technical expertise

3.2. Intended Users (Target Audience):

- Retail business owners (sundry shops, hardware stores, specialty retailers)
- Restaurant and food vendor owners managing perishable daily stock
- Entrepreneurs and early-stage founders with growing product catalogues
- Inventory managers at SMEs without dedicated data analytics resources
- E-commerce sellers managing stock replenishment cycles

© UMHackathon 2026

4.0 System Functionalities

4.1. Description: RetOps operates as a demand prediction and inventory optimisation tool. Users upload transactional data (CSV or Excel) and optionally inventory stock data. The system pre-processes the uploaded files, passes enriched context to the GLM reasoning engine, and returns demand forecasts, reorder recommendations, and scenario-adjusted predictions — all with GLM's reasoning chain made visible to the user. The system integrates weather data and a public holiday calendar to automatically enrich forecast context.

4.2. Key Functionalities:

- **Demand Forecasting (Core Feature) :** GLM analyses the last 30 days of uploaded transactional data in combination with day-of-week patterns and current weather signals to predict per-item demand for the next 7 days. Output includes a visualised forecast chart and GLM's written reasoning so business owners understand the analytical basis of each prediction.

- **Inventory Optimisation :** Available when the user uploads inventory data alongside transactional data. GLM interprets the uploaded CSV or Excel file regardless of format inconsistencies — differing column names, missing headers, or mixed data types — demonstrating the product's unstructured data handling capability. The system identifies which products have the highest sales velocity and flags items at risk of stockout relative to the demand forecast.

- **Reorder Recommendation:** GLM tells users what to buy, how much, and when, with reasoning explained in plain English. Based on forecast vs current stock, GLM outputs a shopping list with quantities and explains why each item is flagged
- **What-If Scenario:** Based on input like "What if tomorrow is a public holiday / it's raining / Ramadan starts." GLM adjusts recommendations accordingly.

4.3 AI Model & Prompt Design

4.3.1. Model Selection

Primary Reasoning Engine Identification

The core reasoning engine powering both the data intelligence and forecasting pipelines is the Z Artificial Intelligence Generative Language Model, specifically utilizing the `ilmu-glm-5.1` foundational model. This specific Large Language Model was explicitly selected over alternative foundational models due to its distinct architectural proficiencies in structured data extraction, strict instruction adherence, and advanced analytical reasoning. These capabilities align perfectly with the zero-trust, automated validation architecture required for the product's primary functionalities.

Capabilities Supporting Data Ingestion and Normalization

For the data ingestion pipeline, the product requires a model capable of parsing highly unstructured and noisy retail transaction logs, as well as disparate spreadsheet uploads, into a rigidly defined JavaScript Object Notation schema. The selected Generative Language Model demonstrates exceptional reliability in zero-shot schema adherence, particularly when configured to operate at a deterministic temperature of zero. Its robust context comprehension allows it to accurately map messy, real-world data fields to a normalized target schema without fabricating or hallucinating missing information. This deterministic extraction capability is the foundational requirement that allows the system's aggressive gatekeeper architecture to function effectively without human intervention.

Capabilities Supporting Demand Forecasting and Analysis

In the context of the demand prediction pipeline, the core product requirement extends beyond basic text transformation into complex time-series analysis and mathematical reasoning. The Z Artificial Intelligence Generative Language Model possesses a highly developed capacity for sequential, chain-of-thought reasoning. By instructing the model to explicitly document its analytical logic prior to generating a final numerical forecast, the system leverages the model's inherent ability to identify historical trends, account for seasonal anomalies, and calculate

© UMHackathon 2026

relational probabilities based on contextual timeframes. This analytical depth ensures that the resulting predictions are grounded in the provided data rather than arbitrary generation.

4.3.2. Prompting Strategy

Zero-Shot Deterministic Prompting

The primary prompting methodology implemented across the system architecture is zero-shot deterministic prompting. In both the data ingestion and demand forecasting modules, the artificial intelligence is provided with a comprehensive system prompt that meticulously details the persona, the critical operating constraints, and the exact JavaScript Object Notation target schema. The architecture deliberately avoids few-shot prompting, which involves providing historical sample inputs and corresponding expected outputs. The justification for this exclusion is the highly variable and unpredictable nature of the input data. Retail transaction logs and historical time-series data encompass an infinite array of formats, item nomenclatures, and categorical structures. Providing a limited set of examples could inadvertently bias the reasoning engine, causing it to overfit to the provided samples rather than utilizing its generalized analytical capabilities to parse novel formats. By coupling highly explicit zero-shot instructions with a temperature parameter strictly set to zero, the system forces the Large Language Model to prioritize rigid structural compliance and deterministic data extraction over creative extrapolation.

Multi-Step Agentic Self-Correction

When the initial zero-shot execution fails to pass the strict internal validation layers, the system dynamically transitions into a multi-step agentic prompting strategy. This approach manifests as an automated, artificial intelligence-driven self-correction loop. If the validation gatekeeper detects a structural anomaly, a mathematical hallucination, or a logical impossibility within the initial output, the system captures the precise programmatic error message generated by the validation layer. The system then programmatically constructs a secondary prompt that explicitly feeds this exact error context back to the reasoning engine. This subsequent prompt commands the

© UMHackathon 2026

model to analyze its previous output, recognize the specific parameter that violated the constraints, and rectify the mistake.

Strategic Justification and Fitness for Use Case

This dual-layered prompting strategy is optimally suited for the core functionalities of the product because it mathematically balances operational efficiency with absolute data integrity. The initial zero-shot pass ensures that the vast majority of standard inputs are processed rapidly with minimal token consumption, thereby optimizing system latency and reducing computational overhead. Simultaneously, the agentic self-correction fallback guarantees that complex, messy, or highly unstructured inputs, which might initially cause the model to falter, are systematically resolved through iterative debugging. This precise combination of zero-shot extraction and agentic refinement ensures that the aggressive gatekeeper architecture functions seamlessly. It maintains a zero-trust data environment and consistently produces reliable supply chain analytics without ever requiring a human escalation path for exception handling.

4.3.3. Context and Input Handling Strategy

Unstructured and Semi-Structured Input Normalization

The system processes unstructured and semi-structured inputs, such as raw text logs, Comma-Separated Values, and Microsoft Excel spreadsheets, through a dedicated normalization middleware before interacting with the artificial intelligence engine. In the data ingestion pipeline, uploaded files are parsed utilizing data manipulation libraries, transformed into structured data frames in memory, and subsequently serialized into a standardized comma-separated string format. This approach leverages the inherent proficiency of Large Language Models in parsing tabular text. In the demand forecasting pipeline, raw database transactions are programmatically pre-aggregated by item name and chronological period into a structured dictionary before being converted to a JavaScript Object Notation string. This aggressive pre-processing step significantly compresses the data payload, stripping unnecessary metadata and providing the reasoning engine

with a highly optimized context format that minimizes token consumption while maximizing analytical focus.

Maximum Input Size and Chunking Philosophy

Regarding maximum input capacity, the system architecture deliberately avoids arbitrary truncation or automated chunking mechanisms. In the context of financial transaction ingestion and supply chain demand forecasting, truncating a dataset or splitting it into disparate chunks would inherently sever relational context and result in data loss. This would lead to skewed analytics, incomplete aggregations, and fundamentally flawed artificial intelligence predictions. Therefore, the maximum input size of the system is strictly bound by the maximum token context window threshold of the underlying reasoning engine Application Programming Interface. The application requires the reasoning engine to process the holistic dataset within a single prompt execution to maintain absolute relational integrity across all historical data points.

Oversized Input Rejection Behavior

When an input payload, such as a massive transaction log or a highly extensive historical dataset, exceeds the maximum context window of the reasoning engine, the system enforces a strict rejection protocol rather than attempting to process partial data. If the input size triggers a payload limit exception from the artificial intelligence Application Programming Interface, the reasoning engine wrapper immediately catches the failure. Consistent with the strict gatekeeper philosophy established throughout the architecture, the system does not attempt to salvage or truncate the oversized request. Instead, the Application Programming Interface endpoints immediately abort the process and return a Hypertext Transfer Protocol error to the client, effectively discarding the transaction. This guarantees that oversized files are explicitly rejected in their entirety, ensuring that partial, fragmented, or incomplete datasets are never committed to the database or utilized for critical supply chain forecasting.

4.3.4. Fallback and Failure Behavior Strategy

© UMHackathon 2026

Architectural Philosophy

To ensure absolute data integrity, the system employs a zero-trust approach to outputs generated by the Large Language Model reasoning engine. Because the data pipelines, specifically transaction ingestion and demand forecasting, are highly sensitive to corrupted or hallucinated data, the system strictly relies on automated deterministic validation rather than human escalation. There is no human-in-the-loop fallback mechanism. If an output cannot be algorithmically verified as completely structurally and logically sound, the system executes a hard rejection. The fallback and failure mechanisms are built upon four primary layers of defense across both the ingestion and forecasting pipelines.

Strict Schema Enforcement

The first layer of defense involves strict schema enforcement at the data type level. The system utilizes validation libraries to act as a strict schema gatekeeper. The JavaScript Object Notation output from the artificial intelligence is not simply parsed into a generic dictionary. Instead, it is forcefully cast against predetermined data models. If the reasoning engine hallucinates a data type, such as returning a text string instead of an integer for a quantity field, the validation layer immediately raises an error. This action prevents the corrupted data from progressing to the database layer.

Domain Logic Validation

Beyond basic data types, the system utilizes custom validators to catch logical and mathematical hallucinations at the domain logic level. In the data ingestion pipeline, a mathematical validator cross-checks the calculations performed by the Large Language Model. This ensures that the total value perfectly matches the product of the quantity and the unit price, while accounting for minor floating-point imprecisions. In the demand forecasting pipeline, a logic validator ensures the model does not predict physically impossible scenarios. This validator explicitly rejects negative forecasted quantities and enforces strict bounds on the confidence score, requiring the value to remain strictly between zero and one.

© UMHackathon 2026

Automated Self-Correction

When a validation error occurs, the system does not immediately fail. Instead, it enters an automated self-correcting retry loop that is configured to a maximum of three attempts. The exact error context generated by the validation layer is dynamically injected back into the subsequent prompt. During these cycles, the reasoning engine operates at a temperature of zero to maximize deterministic and corrective behavior. This configuration forces the artificial intelligence to recognize and correct its specific formatting or logical mistake based on the provided error context.

Graceful Error State

If the artificial intelligence fails to generate a fully compliant response after exhausting all internal retries, the system defaults to a safe and graceful error state. The reasoning engine aborts the transaction entirely and returns a null state. The Application Programming Interface endpoints intercept this state and immediately return a Hypertext Transfer Protocol 422 Unprocessable Entity error to the client. Consequently, the database remains entirely untouched. The system logs the failure for developer visibility, utilizing specific error codes to indicate low confidence or forecast failures, but strictly prevents corrupted inputs or hallucinated demand forecasts from poisoning the downstream business logic.

© UMHackathon 2026

5. User Stories & Use Cases

- **User Stories:**
  - As a retail business owner, I want to upload my transaction history so that RetOps can tell me which items to restock before the week starts, without me having to analyse the data myself.
  - As a food vendor, I want to receive a shopping list with quantities and plain-language explanations so that I can make procurement decisions confidently without second-guessing.
  - As a restaurant owner, I want to enter a scenario like "Hari Raya is in 3 days" so that the system adjusts its recommendations to reflect the expected demand surge.
  - As a business owner with inconsistently formatted spreadsheets, I want the system to read my files regardless of how the columns are named so that I do not need to reformat my data manually before uploading.
  - As a retail business owner with limited AI knowledge, I want the system to explain its reasoning in Malay or English so that I understand why it is recommending what it recommends.

- **Use Cases (Main Interactions):**

  Data Upload Flow

  User uploads transactional CSV/Excel (mandatory) and optionally inventory stock data via the web interface. The backend parses and validates the files using pandas, flags any structural issues with a user-visible warning, and confirms successful

© UMHackathon 2026

ingestion. GLM normalises inconsistent column structures before any analysis begins.

Demand Forecast Flow

The system extracts the last 30 days of sales records, applies day-of-week weighting, fetches current weather data from OpenWeatherMap, and checks the public holiday calendar. This enriched context is assembled into a structured prompt and passed to GLM. The model returns a 7-day per-item demand forecast with written reasoning. The frontend renders a forecast chart alongside GLM's explanatory text.

Reorder Recommendation Flow

GLM compares the forecasted demand against current inventory levels (if uploaded) and generates a prioritised shopping list. Each line item specifies: item name, quantity to order, recommended order date, and a plain Malay/English justification. The list is displayed as an actionable table the owner can use directly for procurement.

What-If Scenario Flow

User enters a free-text contextual scenario into the scenario input field. GLM re-processes the forecast and recommendation pipeline with the scenario as additional context. The updated output is displayed with a highlighted explanation of what changed relative to the baseline forecast and why the scenario caused those adjustments.

- **Use Case Diagram:** *(Optional but encouraged for participants to visualize the interaction).*

© UMHackathon 2026

6. Features Included (Scope Definition)

- Conversational agent for retail business owners to optimize their inventory and predict future demand
- CSV and Excel file upload for transactional data (mandatory) and inventory stock data (optional)
- GLM-powered unstructured data interpretation — handles column naming inconsistencies and mixed data formats without manual reformatting by the user
- 7-day per-item demand forecast with visualisation and GLM's written reasoning output
- AI-generated reorder shopping list with item, quantity, timing, and plain English explanations per line item
- Weather context integration via OpenWeatherMap API to enrich demand forecast accuracy
- Public holiday calendar integration to automatically flag demand-impacting events
- What-If Scenario input for contextual demand adjustment and trade-off analysis
- Conversational agent interface for retail owners to query insights via text prompt

© UMHackathon 2026

Email: umhackathon@um.edu.my

7. Features Not Included (Scope Control)

- Real-time POS integration (if not implemented)
- Automated ordering system
- Advanced ERP integration

© UMHackathon 2026
Email: umhackathon@um.edu.my

8. Assumptions & Constraints

- **LLM Cost Constraint:** Token consumption per average user session is estimated at 3,000–5,000 tokens, capped at 5,000 tokens per request via limits.py. To manage inference costs at scale, repeated identical prompts within the same session are cached and not re-processed. Data context is compressed via pre-aggregation before injection — raw rows are never passed to GLM. Prompt templates use fixed-length structures to prevent unbounded token growth.
- **Technical Constraints:**
  - Data upload only support CSV and Excel format. GLM are responsible for reasoning, not just formatting
  - OpenWeatherMap free tier api key offers survive that is sufficient for demo, not for production
  - Only covers Malaysian cities in your CITY_COORDS dict — any city outside that list defaults to KL silently

- **Performance Constraints:**
  - High quality response requires more tokens
  - Prediction generation time increases with dataset size
  - System is optimized for batch processing, not real-time forecasting
  - Visualization complexity is limited to ensure responsiveness
  - Handling extremely large datasets may result in delays or require preprocessing

- **LLM / AI Constraints:**
  - The system uses predefined forecasting models
  - Token cost per request are estimated and capped at 5k tokens per request
  - limits.py limits user to prompt for 8000 characters maximum
  - High-quality explanations require more tokens, increasing cost
  - Responses are limited by context window size

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **User Input:** The system requires a minimum of 7 days of transactional records to generate a meaningful demand forecast. Sessions with insufficient data will display an informative error. All GLM outputs require human review, the system does not make autonomous procurement decisions. The GLM cannot generate forecasts without an uploaded dataset as the primary input context.

© UMHackathon 2026

9. Risks & Questions Throughout Development

- **Data Quality Risk**

  If uploaded transactional data contains significant gaps, duplicate entries, or entirely non-standard date formats that pandas cannot parse, GLM's forecast accuracy may degrade without the user being aware. Mitigation: implement a data quality pre-check step that surfaces a warning to the user before forecast processing begins, quantifying the percentage of rows that were dropped or flagged.

- **Forecast Confidence for Low-Volume Items**

  For items with fewer than 7 historical data points, the 7-day forecast has insufficient basis and may be unreliable. These items should be explicitly flagged in the output as low-confidence predictions rather than presenting them with equal weight to high-data items.

- **Scenario Interpretation Ambiguity**

  Vague or ambiguous What-If inputs — such as "big event tomorrow" — may be interpreted differently by GLM than the user intends, producing unexpected recommendation changes. Mitigation: provide guided scenario examples in the UI and have GLM explicitly state its interpretation of the scenario at the top of its output before showing adjusted recommendations.

- **Hallucination Risk in Reorder Quantities**

  GLM may generate plausible-sounding but numerically incorrect reorder quantities. The domain logic validator must include a sanity check that flags quantities exceeding 3× the 30-day historical average for any single item, preventing implausible recommendations from reaching the user without a warning.

- **Weather API Dependency**

© UMHackathon 2026
Email: umhackathon@um.edu.my

The OpenWeatherMap free tier may rate-limit during peak usage. If weather data is unavailable, the system should proceed with forecast generation while noting the absence of weather context in the output, rather than failing the entire request.

© UMHackathon 2026

Email: umhackathon@um.edu.my